

Software Requirements Specification (SRS) and Validation Specification

Project 39: Dehallucination Add-On Module ("Dehallucinator")

UE24CS341A Software Engineering, Jackfruit Mini-Project

PES University

Version 1.0

5 October 2026

Team

Sai Nikitha Perumalla (Role A, Claim Extractor)

Shreya Raghuraj (Role B, Evidence Retriever)

Ralph D'souza (Role C, Verifier & Scorer)

Satwik (Role D, Corrector & Integrator)

1. Introduction

1.1 Purpose: This document specifies the requirements for Dehallucinator, a module that sits between a chatbot and the user, checks the chatbot's answer against trusted sources, and corrects or flags wrong parts.

1.2 Scope: The system extracts claims from an answer, retrieves evidence, verifies each claim with an NLI model, gives a trust score, and returns a corrected answer through a REST API and a demo UI.

1.3 Definitions:

- **Claim:** one standalone factual statement.
- **Evidence:** a snippet from Wikipedia or the local KB.
- **Verdict:** SUPPORTED, CONTRADICTED or NOT_ENOUGH_INFO.
- **KB:** the 15 trusted text files in data/kb/.
- **NLI:** natural language inference.

1.4 References: Project 39 Team Manual; MediaWiki API documentation; model card for cross-encoder/nli-deberta-v3-xsmall.

2. Non-Functional Requirements

The six NFRs are defined once here. Each module in Section 3 lists which of them apply to it.

ID	Requirement	How it is checked
NFR-1	Performance: A 300-word answer is fully checked in ≤ 20 s on a normal laptop CPU (Wikipedia results cached)	System test TC-SYS03
NFR-2	Reliability: If Wikipedia is down, the system still returns a result using the local KB and does not crash	TC-2-08
NFR-3	Security: Input is limited to 5000 chars and validated; bandit reports no High issues; pip-audit reports no known critical vulnerabilities; no secrets in the repo	Jenkins security stage + security tests
NFR-4	Maintainability: Line + branch coverage $\geq 80\%$ per module; ruff clean; SonarQube Quality Gate passed	Jenkins + SonarQube
NFR-5	Portability: Runs with one command via Docker	Jenkins deploy stage
NFR-6	Usability: The UI marks each claim green/red/grey and shows its source link	System test TC-SYS05

3. Functional Requirements

Each subsection covers one module and follows the same format: Overview, Use Case, Functional Requirements, Non-Functional Requirements, Limitations. Use cases are UC-1 to UC-4, and functional requirements are FR-1.x to FR-4.x, in the same order as the modules.

3.1 Claim Extractor (Owner: Sai Nikitha Perumalla, Role A)

Overview

The Claim Extractor is the first stage of the pipeline. It takes an AI-generated answer and splits it into standalone factual claims so that each one can be checked separately. It removes opinions,

questions and filler, splits compound sentences, and replaces pronouns so each claim makes sense on its own.

Interface: `extract_claims(text: str) -> list[Claim]`, where `Claim = {id, text, sentence_index}`

Use Case UC-1: Extract factual claims from an AI answer

Actor: Dehallucination pipeline

Preconditions: An AI answer is available as plain text, and the spaCy `en_core_web_sm` model is installed.

Main flow:

1. The pipeline sends the AI answer to the Claim Extractor.
2. The system checks that the text is not empty and is at most 5000 characters.
3. The system splits the text into sentences.
4. The system removes non-factual sentences.
5. The system splits compound sentences into separate claims.
6. The system replaces leading pronouns with the last named entity.
7. The system removes duplicate claims.
8. The system numbers the claims and returns them.

Alternate/error flows:

- Empty input or more than 5000 characters: a `ValueError` is raised and the API returns HTTP 422.
- No factual sentences: an empty list is returned.

Postconditions: A list of numbered Claim objects is returned to the pipeline.

Related FRs: FR-1.1 to FR-1.7

Functional Requirements

- **FR-1.1:** The system shall reject input that is empty or longer than 5000 characters.
- **FR-1.2:** The system shall split the input text into sentences.
- **FR-1.3:** The system shall discard non-factual sentences: questions, sentences with opinion words (I think, maybe, probably, best, should, etc.), sentences starting with "I" or "We", and sentences under 4 words.
- **FR-1.4:** The system shall split a compound sentence at ";", ", and" or ", but" only when both parts have their own subject and verb.
- **FR-1.5:** The system shall replace a leading pronoun (He, She, It, They, etc.) with the most recent named entity from earlier sentences.
- **FR-1.6:** The system shall remove duplicate claims, ignoring case and punctuation.
- **FR-1.7:** The system shall number claims from 1 in order and record the index of the sentence each came from.

Non-Functional Requirements

- **NFR-1:** The module extracts claims from a 300-word answer in under 1 second (module target within the 20 s overall limit).
- **NFR-3:** Input is validated and limited to 5000 characters (FR-1.1).
- **NFR-4:** Unit test coverage of the module is at least 80%.

Limitations

- English only.
- Pronoun replacement is rule-based and may pick the wrong entity in complex paragraphs.

3.2 Evidence Retriever (Owner: Shreya Raghuraj, Role B)

Overview

The Evidence Retriever is the second stage of the pipeline. For each claim, it searches Wikipedia and the local KB of 15 documents, ranks the snippets by TF-IDF cosine similarity, and returns the top 3. Wikipedia pages are cached in SQLite. If Wikipedia is unreachable, it uses the KB only.

Interface: `retrieve_evidence(claims: list[Claim]) -> list[Evidence]`, where `Evidence = {claim_id, source, source_url, title, snippet, relevance}` and `source` is "wikipedia" or "local_kb".

Use Case UC-2: Retrieve evidence for a claim from trusted sources

Actor: Pipeline

Preconditions: A list of Claim objects is available, the spaCy model is installed, and the KB files are in `data/kb/`.

Main flow:

1. The pipeline sends the claims to the Evidence Retriever.
2. The system loads the KB files once.
3. For each claim, the system builds a search query from named entities, then noun chunks (max 8 words).
4. The system searches Wikipedia for the top 3 pages, using cached text if it is under 7 days old.
5. The system splits the Wikipedia and KB text into snippets (a sentence plus the next one).
6. The system ranks the snippets by TF-IDF cosine similarity with the claim and keeps the top 3.
7. The system returns Evidence objects linked to each claim by `claim_id`.

Alternate/error flows:

- Wikipedia times out or is down: after 2 retries (5 s timeout), a warning is logged and the KB alone is used.
- No evidence found: no Evidence is returned for that claim.
- Empty claim list: an empty list is returned.

Postconditions: A list of Evidence objects is returned to the pipeline.

Related FRs: FR-2.1 to FR-2.8

Functional Requirements

- **FR-2.1:** The system shall build a search query per claim from named entities, then noun chunks, without stop words, in at most 8 words.
- **FR-2.2:** The system shall search Wikipedia through the MediaWiki API (top 3 results) and send a User-Agent header.
- **FR-2.3:** The system shall fetch the page text and keep the first 6000 characters.
- **FR-2.4:** The system shall read all `data/kb/*.txt` files once at start-up and treat each as a page with source "local_kb".

- **FR-2.5:** The system shall split pages into snippets and rank them by TF-IDF cosine similarity with the claim, returning the top 3 across both sources, with relevance equal to the cosine score.
- **FR-2.6:** The system shall cache Wikipedia pages in SQLite and reuse them if they are under 7 days old.
- **FR-2.7:** The system shall not crash on network failure. It shall retry twice with a 5 s timeout, log a warning and continue with the local KB.
- **FR-2.8:** The system shall return no Evidence for a claim when nothing is found.

Non-Functional Requirements

- **NFR-1:** Wikipedia results are cached so a 300-word answer is checked within the 20 s limit.
- **NFR-2:** If Wikipedia is down, the retriever uses the local KB and does not crash (checked by TC-2-08).
- **NFR-3:** SQL-like input must not break the cache.
- **NFR-4:** Unit test coverage of the module is at least 80%.

Limitations

- Wikipedia needs an internet connection, and its content can change.
- The KB covers only 15 topics.
- TF-IDF matches words, not meaning, so paraphrased facts can be missed.
- Only the first 6000 characters of a Wikipedia page are used.

3.3 Verifier & Scorer (Owner: Ralph D'souza, Role C)

Overview

The Verifier & Scorer is the verification stage of the pipeline. It checks each extracted claim against its available evidence using an NLI model, determines whether the claim is supported, contradicted, or unverifiable, and calculates an overall trust score for the answer.

Interface: `verify_claims(claims: list[Claim], evidence: list[Evidence]) -> list[Verdict]` and `trust_score(verdicts: list[Verdict]) -> float`

Use Case UC-3: Verify a claim against its evidence and score the answer

Actor: Pipeline

Preconditions: Extracted claims and their corresponding evidence are available, and the NLI model is loaded.

Main flow:

1. The pipeline sends a claim and its evidence to the Verifier & Scorer.
2. The system loads the NLI model if it is not already loaded.
3. The system scores each claim-evidence pair using the NLI model.
4. The system assigns a verification label based on the NLI result.
5. The system applies the number/year mismatch rule when applicable.
6. The system calculates the trust score for the answer.
7. The system returns the verification results and trust score to the pipeline.

Alternate/error flows:

- No evidence: the claim is marked as NOT_ENOUGH_INFO without calling the model.
- Number/year mismatch: the claim is forced to CONTRADICTED if the evidence contains a conflicting number.

Postconditions: A list of Verdict objects and an overall trust score are returned to the pipeline.

Related FRs: FR-3.1 to FR-3.6

Functional Requirements

- **FR-3.1:** The system shall load the cross-encoder/nli-deberta-v3-xsmall model once at the module level (lazy loading).
- **FR-3.2:** The system shall score claim-evidence pairs as one batch inside `torch.no_grad()`, calculating softmax probabilities for entailment, contradiction, and neutral.
- **FR-3.3:** The system shall assign SUPPORTED if entailment $\geq$ SUPPORT_THRESHOLD and entailment $\geq$ contradiction. It shall assign CONTRADICTED if contradiction $\geq$ CONTRADICT_THRESHOLD. Otherwise, it shall assign NOT_ENOUGH_INFO.
- **FR-3.4:** If no evidence is found, the system shall return NOT_ENOUGH_INFO with a score of 0.0 and best_evidence as None, without calling the model.
- **FR-3.5:** The system shall extract numbers using the regex `\d[\d, .]*`. If both the claim and best snippet mention the same named entity and contain numbers, but the claim's number is missing from the snippet, it shall force CONTRADICTED with a score of $\max(\text{contradiction}, 0.75)$.
- **FR-3.6:** The system shall calculate the trust score as $100 \times \text{average of (SUPPORTED} = 1, \text{NOT_ENOUGH_INFO} = 0.5, \text{CONTRADICTED} = 0)$, rounded to 1 decimal place. It shall return 100 if there are no claims.

Non-Functional Requirements

- **NFR-2:** Verification does not crash if Wikipedia network calls fail. The claim is handled as having no evidence (FR-3.4).
- **NFR-4:** Unit test coverage of the module is at least 80%.

Limitations

- Verification accuracy depends entirely on the quality of the retrieved Wikipedia/KB snippets.
- The number mismatch rule relies on exact regex extraction and may miss spelled-out numbers (e.g., "five" vs "5").

3.4 Corrector & Integrator (Owner: Satwik, Role D)

Overview

The Corrector & Integrator is the final stage of the pipeline and the part other applications use. It rewrites the AI answer using the verdicts from the Verifier: wrong claims are replaced with a correction and its source, unverifiable claims are flagged, and supported claims are tagged with their source. It also connects the four stages into one `check_answer()` call and exposes it through a REST API, a one-line chatbot decorator and a demo UI.

Interfaces: `correct(text: str, claims: list[Claim], verdicts: list[Verdict]) -> str`; `check_answer(text: str) -> CheckedAnswer`, where `CheckedAnswer = {original, corrected, claims, verdicts, trust_score}`; the `@dehallucinate` decorator; and the REST API POST `/check` and GET `/health`.

Use Case UC-4: Check and correct a chatbot answer

Actor: Chatbot developer (through the REST API or the @dehallucinate decorator); end user (through the demo UI).

Preconditions: The dehallucinator API, library or UI is running, and UC-1, UC-2 and UC-3 are available.

Main flow:

1. The actor submits an AI answer (POST /check, a decorated function's return value, or the UI "Check" button).
2. The system checks that the text is not empty and is at most 5000 characters.
3. The system extracts the claims (UC-1).
4. The system retrieves evidence for each claim (UC-2).
5. The system verifies each claim and calculates the trust score (UC-3).
6. The system builds the corrected answer from the verdicts.
7. The system returns a CheckedAnswer to the actor.
8. (UI only) The system shows each claim colour-coded with its source link, the trust score and the corrected answer.

Alternate/error flows:

- Invalid input: missing, empty or whitespace-only text, wrong type, more than 5000 characters, or malformed JSON is rejected with HTTP 422 and nothing is checked.
- No factual claims: the corrected answer equals the original, the claim and verdict lists are empty, and the trust score is 100.
- Evidence sources unavailable: claims without evidence are NOT_ENOUGH_INFO (FR-3.4) and are flagged [unverified] in the corrected answer.
- Unexpected internal error: the error is logged and HTTP 500 is returned with a generic message and no stack trace.

Postconditions: The actor receives a CheckedAnswer with the original text, the corrected text, the claims, one verdict per claim and a trust score from 0 to 100.

Related FRs: FR-4.1 to FR-4.6

Functional Requirements

- **FR-4.1:** The system shall build the corrected answer by rewriting each claim according to its verdict:
 - (a) a CONTRADICTED claim with evidence is replaced by the evidence snippet followed by "[corrected, source: <url>]";
 - (b) a NOT_ENOUGH_INFO claim is kept and followed by "[unverified]";
 - (c) a SUPPORTED claim is kept and followed by "[source: <url>]";
 - (d) a CONTRADICTED claim without evidence, or a claim with no verdict, is treated as NOT_ENOUGH_INFO;
 - (e) text that is not part of a claim is left unchanged;
 - (f) if a claim's text does not appear in its sentence (for example after pronoun replacement, FR-1.5), the tag is added at the end of the sentence given by sentence_index, and a CONTRADICTED claim that is the only claim in that sentence replaces the whole sentence.
- **FR-4.2:** The system shall provide check_answer(text), which runs extract, retrieve, verify, score and correct in that order, returns a CheckedAnswer, logs the time taken by each

stage in milliseconds, and raises `ValueError` for empty input or input longer than 5000 characters.

- **FR-4.3:** The system shall provide a `@dehallucinate` decorator that wraps any function returning a `str` so that the wrapped function returns a `CheckedAnswer` for that string, keeps the original function's name and docstring, and raises `TypeError` if the function does not return a `str`.
- **FR-4.4:** The system shall expose a REST API with `POST /check` and `GET /health` as described under External Interfaces, return HTTP 422 for invalid input, and return HTTP 500 with a generic message and no stack trace for unexpected errors.
- **FR-4.5:** The demo UI shall let the user pick one of 10 sample answers from a drop-down (loaded from `data/samples.json`, on topics covered by the KB) or paste an answer of up to 5000 characters, and check it with one button.
- **FR-4.6:** The demo UI shall show each claim coloured green (SUPPORTED), red (CONTRADICTED) or grey (NOT_ENOUGH_INFO) with its source link, the trust score and the corrected answer, and shall HTML-escape all answer text so that no HTML or script from the input is rendered.

External Interfaces

- **User interface:** Streamlit web app (`streamlit run ui/app.py`) providing the views in FR-4.5 and FR-4.6.
- **Python library:** `check_answer` from `dehallucinator.pipeline`, and the `dehallucinate` decorator from `dehallucinator.guard`.
- **REST API:** FastAPI app `dehallucinator.api:app`, served with `uvicorn`. Request and response bodies are JSON.
 - `POST /check`: Request `{"text": "<AI answer>"}`, where text is required, 1 to 5000 characters and not only whitespace. Returns 200 with the `CheckedAnswer` as JSON; 422 for invalid input or malformed JSON, with `{"detail": [...]}`; 500 with `{"detail": "Internal server error"}`.
 - `GET /health`: Returns 200 with `{"status": "ok"}`. Used by Docker and Jenkins.
- **Example:** `POST /check` with "The Eiffel Tower was completed in 1899. It is located in Paris." returns the claims "The Eiffel Tower was completed in 1899." (CONTRADICTED) and "The Eiffel Tower is located in Paris." (SUPPORTED), a `trust_score` of 50.0, and the corrected answer "The Eiffel Tower was completed in 1889. [corrected, source: https://en.wikipedia.org/wiki/Eiffel_Tower] It is located in Paris. [source: https://en.wikipedia.org/wiki/Eiffel_Tower]".

Non-Functional Requirements

- **NFR-3:** The API validates all input and never returns a stack trace.
- **NFR-4:** Unit test coverage of the module is at least 80%.
- **NFR-5:** The API runs with one command via Docker.

Limitations

- A correction is only as good as the best evidence snippet: the snippet replaces the wrong claim as it is and may contain extra detail.
- The API has no authentication or rate limiting, so it is meant for local and demo use.
- The demo UI checks one answer at a time.

4. Validation Approach

- **Unit testing:** Test individual verification logic using mocked model probabilities to prevent actual model downloads during unit tests.
- **Integration testing:** Verify data flows correctly from B (Evidence) to C (Verifier).
- **System testing:** Start the API with FastAPI TestClient, send all 10 sample answers, and verify known hallucinations are flagged as CONTRADICTED while timing the response.
- **Security testing:** Ensure inputs exceeding limits or containing malformed JSON/HTML return a 422 error and do not break the system.
- **Testing tools:** pytest, pytest-cov (for $\geq 80\%$ line and branch coverage), and SonarQube Cloud (Quality Gate).